Are Stagnation Signals Structural? A Systematic Empirical Study of Progress Detection in LLM Agents via Weak Supervision

Anonymous ACL submission

LLM-based agents deployed on software engineering tasks frequently exhibit semantic stagnation—executing diverse yet collectively unproductive action sequences that extend beyond simple n-gram repetition and overwhelm LLM-based judges. No systematic study has compared detection approaches across deployment scenarios or investigated what signals carry stagnation information.

We propose a weak supervision methodology for agent stagnation monitoring: three labeling functions aggregated by majority vote produce silver labels over 205K+ windows from two coding agents (SWE-Agent and OpenHands) on SWE-bench. Against both silver labels (LF-agreement scores) and a human-annotated gold set (N=505, three annotators), we compare 30 structural features, a GRU-based neural probe, n-gram cycle detectors, and LLM-as-judge baselines across zero-data and fine-tuned regimes.

We find a deployment-regime trade-off. On a human-annotated gold set (N=505), the best LLM judge achieves the highest non-circular accuracy (few-shot-CoT 0.910 vs. structural GB 0.839), but LLM judges collapse on cross-agent transfer (0.491 vs. 0.967, −47.6 pp) at orders-of-magnitude higher cost. Cycle detection matches structural features on in-domain gold (∼0.84) but likewise collapses cross-agent (−43.7 pp). In zero-data transfer, structural features outperform the neural probe by 39.4 pp; with fine-tuning, the probe surpasses structural features (0.976 vs. 0.967).

Labeling functions exhibit agent-specific temporal dynamics: command repetition leads stagnation by 1.87 steps on SWE-Agent but lags by 3.32 steps on OpenHands. Label version bias—not LF noise—was the dominant cross-coding-agent transfer bottleneck. Five label models (three temporal, two generative) all fail to improve over majority vote; the bottleneck is LF quantity, not aggregation sophistication.

1 Introduction

LLM agents are increasingly deployed for complex, long-horizon software engineering tasks—resolving GitHub issues, debugging production systems, and iterating on multi-file codebases across dozens of editing rounds (Jimenez et al., 2024; Yang et al., 2024a; Wang et al., 2025a). As these deployments scale, a critical failure mode has emerged that is qualitatively different from crashes or syntax errors: the agent *stagnates*—continuing to produce actions that appear purposeful while making no meaningful progress toward the task goal. Orlanski et al. (2026) document this degradation on SlopCodeBench, recording verbosity increases of 77% and quality drops of 75.5% in long-horizon coding trajectories, with no agent fully solving any benchmark problem end-to-end. Crucially, the dominant failure pattern is not simple repetition but what we term *semantic stagnation*: the agent issues varied commands, edits different files, and generates plausible-looking outputs, yet its behavior is functionally stuck. This "looks busy but going in circles" pattern defeats existing monitoring tools, because surface-level action diversity masks the underlying lack of progress. We observe this phenomenon at scale: across trajectories from two architecturally distinct agents—SWE-Agent (Yang et al., 2024a) and OpenHands (Wang et al., 2025a)—on SWE-bench tasks (Jimenez et al., 2024), 28.5% and 20.9% of sliding windows exhibit stagnation under our silver labeling scheme, respectively, confirming that the failure mode is both prevalent and highly agent-dependent.

Existing approaches address aspects of this problem but fall short in three ways. First, *n-gram cycle detectors* (George et al., 2025; Pipis et al., 2025) flag exact action-sequence repetition and match richer structural features on in-domain gold (∼0.84 macro-F1), but collapse on cross-agent transfer (−43.7 pp; §4.8)—they lack the richer structural vo-

cabulary (fuzzy match ratios, edit distances, file operation counts) needed to generalize across agents. Second, *step-level scoring* via process reward models (Xi et al., 2025) evaluates individual actions in isolation, discarding the window-level temporal dynamics that distinguish transient setbacks from sustained stagnation—a distinction that requires reasoning over sequences of consecutive actions (Chen et al., 2025). A sequence of failed commands followed by a strategy switch, for instance, carries different diagnostic information than the same commands evaluated independently. Third, *LLM self-evaluation* (Shinn et al., 2023) achieves strong in-domain accuracy (0.910 gold macro-F1 with few-shot-CoT) but is costly ($40+ per evaluation) and brittle across agents: the best LLM reaches only 0.491 on cross-agent transfer (§4.3), ruling out deployment as a general-purpose monitor. More fundamentally, none of these approaches asks the prior question: *what signals actually carry stagnation information?* No systematic study has investigated whether detecting subtle agent failure requires sophisticated semantic understanding or whether simpler structural patterns suffice.

We address this question through a systematic empirical study (Figure 1). Using a weak supervision framework (Ratner et al., 2016, 2019a), we construct silver-labeled datasets from over 205K windows spanning SWE-Agent (54,809 windows, 28.5% silver-labeled stagnation) and OpenHands (150,652 windows, 20.9% stagnation). Three labeling functions provide complementary noisy votes: command repetition (λ_1) targets repetitive behavior, file diff delta (λ_2) targets output productivity, and outcome backpropagation (λ_3) propagates task-level outcomes backward through the trajectory. Majority vote aggregation produces window-level binary labels with confidence scores. Our window-level formulation (k=5 consecutive actions, stride 1) captures short-range temporal patterns—such as repeated failures followed by strategy changes—that step-level approaches cannot express, while providing the granularity to pinpoint *when* stagnation begins and ends within a trajectory, enabling targeted interventions. Against these silver labels, we systematically compare hand-crafted structural features, a neural embedding probe, and an LLM-as-judge baseline.

Our study yields three principal findings.

Finding 1: Structural features uniquely enable zero-data cross-agent transfer. On gold, the best LLM judge achieves the highest non-circular accuracy (0.910 vs. structural GB 0.839), but LLM judges collapse on cross-agent transfer (0.491 vs. 0.967, -47.6 pp). Cycle detectors match structural GB on in-domain gold ($\sim$0.84) but likewise collapse cross-agent (-43.7 pp). In zero-data transfer, structural features outperform the probe by 39.4 pp; with fine-tuning, the probe surpasses structural features (0.976 vs. 0.967).

Finding 2: Label quality, not LF selection, is the dominant transfer bottleneck. LFs exhibit temporal dynamics that reverse across agents (λ_1 leads by -1.87 steps on SWE-Agent, lags by $+3.32$ on OpenHands). Fixing OH label version bias (stag rate 6.56%$\rightarrow$21.4%) improves 5% fine-tuning F1 from 0.570 to 0.850; λ_1's core value is coverage (-29.4 pp when removed), not precision.

Finding 3: No label model improves over majority vote. Five label models—three temporal and two generative (Safranchik et al., 2020; Fu et al., 2020; Ratner et al., 2019b)—all fail to surpass majority vote on downstream detection. The bottleneck is LF quantity: three binary LFs produce too little inter-LF conflict for any method to exploit.

Our contributions are:

1. **Weak supervision methodology for agent monitoring.** We design a silver-labeling pipeline (3 LFs + majority vote) over 205K+ windows from two coding agents, enabling systematic comparison of detection methods without manual annotation.

2. **Label quality as transfer bottleneck.** We show that labeling functions exhibit agent-specific temporal reversals and that fixing label version bias (stag rate 6.56%$\rightarrow$21.4% on the embedding subset) is the dominant factor for cross-coding-agent transfer, informing future weak-supervision design on sequential agent data.

3. **Label model negative finding.** We show that five label models—three temporal and two generative—fail to improve over majority vote for downstream detection. The bottleneck is LF quantity, not aggregation method: the 3-LF regime produces insufficient inter-LF conflict for any approach to add value.

4. **Deployment-regime trade-off.** On gold, the best LLM judge achieves the highest non-circular accuracy (0.910 vs. structural GB

0.839), but LLM judges collapse on cross-agent transfer (0.491 vs. 0.967, −47.6 pp) at 8,000× higher cost. Structural features uniquely enable zero-data cross-agent transfer, outperforming the probe by 39.4 pp and cycle detectors by 43.7 pp (no GPU, <5 ms per window). With fine-tuning, the probe surpasses structural features (0.976 vs. 0.967), delineating when each approach is warranted.

2 Related Work

Process Supervision and Self-Reflection. PRMs assign step-level quality scores (Xi et al., 2025; Choudhury, 2025; Fan et al., 2026), building on process supervision (Lightman et al., 2024; Uesato et al., 2022), with benchmarks revealing long-horizon degradation (Wang et al., 2026; Li et al., 2026). Self-reflection methods (Shinn et al., 2023; Yao et al., 2023b,a; Zhou et al., 2024a) prompt agents to diagnose failures, but LLMs cannot reliably self-correct without external feedback (Huang et al., 2024; Kamoi et al., 2024). We address a complementary question—*when* is the agent stuck—using external structural signals at window level, targeting real-time monitoring (<5 ms) without outcome labels.

Weak Supervision, Noisy Labels, and Label Leakage. Data programming (Ratner et al., 2016, 2019a) enables dataset creation through labeling functions, extended via multi-task aggregation (Ratner et al., 2019b), triplet methods (Fu et al., 2020), sequential HMMs (Safranchik et al., 2020), multi-resolution LFs (Varma et al., 2019), and benchmarked by WRENCH (Zhang et al., 2021). Zhan et al. (2019) apply WS to multi-agent trajectory generation; our work targets single-agent monitoring. Our application differs from standard Snorkel pipelines in three ways: LFs target agent trajectory structure (command repetition, file diffs) rather than text patterns; labels exhibit temporal dependencies within and across windows; and LF firing patterns are agent-specific, requiring cross-agent transfer analysis absent from prior weak supervision work. Unlike Safranchik et al.'s NER results, temporal label models do not improve over majority vote in our low-conflict 3-LF regime (§4.7). Our silver labels connect to noisy label learning (Han et al., 2018; Li et al., 2020); the structural feature–label overlap instantiates label leakage (Kaufman et al., 2012) and Goodhart's Law (Manheim and Garrabrant, 2018), mitigated through cross-coding-agent evaluation and a gold set (§4.8).

Agent Benchmarks and Monitoring. Agent evaluation spans diverse environments—general tasks (Liu et al., 2024), web navigation (Zhou et al., 2024b), interactive coding (Yang et al., 2024b), tool use (Qin et al., 2024), and debugging (Tian et al., 2024)—creating demand for scalable monitoring. Orlanski et al. (2026) document coding agent degradation; Rath et al. (2026) quantify multi-agent behavioral drift; Chen et al. (2025) show sequence-level context is necessary; Kale et al. (2025) show weak monitors can oversee stronger agents; and Cemri et al. (2025); Zhu et al. (2025) catalog systematic failure modes. Guardrail frameworks (Rebedea et al., 2023; Xiang et al., 2024; Wang et al., 2025b) address safety but not progress.

Anomaly Detection, Process Mining, and Compute Efficiency. Stagnation detection relates to sequential anomaly detection (Chandola et al., 2009), including deep log analysis (Du et al., 2017; Meng et al., 2019), runtime verification (Leucker and Schallhart, 2009), and change point detection (Aminikhanghahi and Cook, 2017; Adams and MacKay, 2007). Cycle detectors (George et al., 2025; Pipis et al., 2025) target structural repetition but miss semantically diverse stagnation; Liu et al. (2026) train trajectory verifiers for broader anomaly detection. Our structural features draw on process mining (van der Aalst, 2016); the success of hand-crafted features parallels classical findings for narrow detection tasks (Liu et al., 2008). Early-exit architectures (Teerapittayanon et al., 2016; Schwartz et al., 2020) and curriculum learning (Bengio et al., 2009) reduce computation by adapting effort to difficulty, analogous to terminating unproductive agent computation.

3 Method

We frame stagnation detection as window-level binary classification over agent trajectories. Silver labels are generated via weak supervision from three labeling functions, then used to train and evaluate three detection approaches: structural features, a neural embedding probe, and an LLM-as-judge baseline. Figure 1 provides a system overview.

3.1 Problem Formulation

Given an agent trajectory $T = (a_1, a_2, \ldots, a_n)$ where each a_t is an action (e.g., a shell command, file edit, or API call), we partition it into overlapping sliding windows $w_i = (a_i, \ldots, a_{i+k-1})$ with

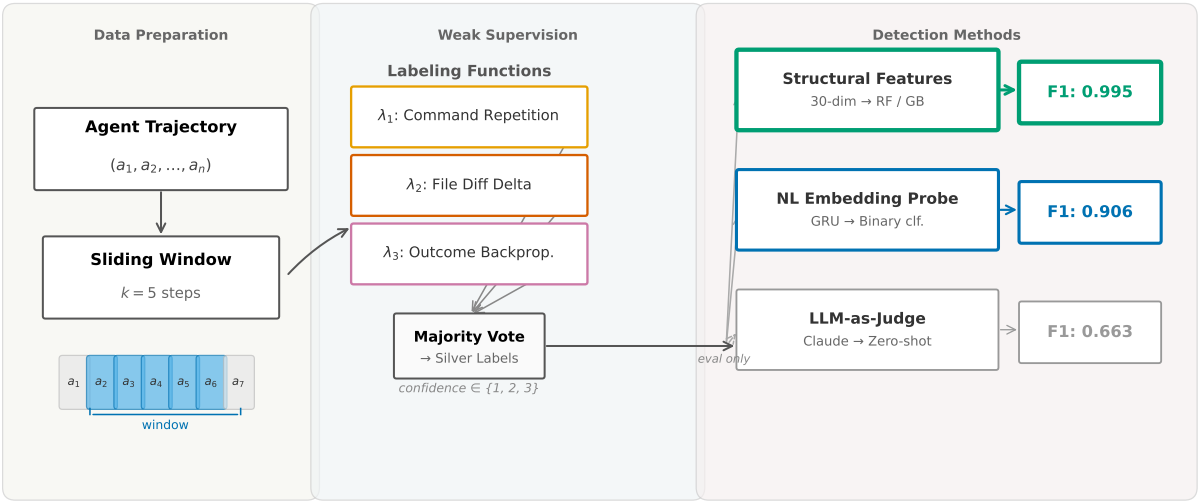


Figure 1: System overview. (Left) Agent trajectories are segmented into overlapping windows of k=5 steps. (Center) Three labeling functions vote on each window; majority vote produces silver labels with confidence 1–3. (Right) Three detection approaches compared: structural features (30-dim), NL embedding probe (GRU), and LLM-as-judge (eval only, not trained on silver labels).

window size k=5 and stride 1. The task is to assign each window a binary label $y_i \in \{0, 1\}$, where 0 denotes *stagnation* (the agent is stuck or unproductive) and 1 denotes *progress* (the agent is making meaningful headway toward the task goal).

The window formulation offers two advantages over alternatives. Compared to step-level scoring, windows capture short-range temporal patterns—such as a sequence of repeated failed attempts—that individual actions cannot express. Compared to trajectory-level labeling, windows provide the granularity needed to pinpoint *when* stagnation begins and ends within a single task attempt, enabling downstream interventions such as early termination or strategy switching.

3.2 Weak Supervision Pipeline

Prior gold-standard annotations for agent stagnation at the window level are unavailable; we construct a stratified gold set (§4.8). Manual annotation is expensive: each window requires reviewing k actions with full context (command text, outputs, file diffs), taking an estimated 2–3 minutes per window. We therefore adopt a data programming approach (Ratner et al., 2016, 2019a): multiple labeling functions (LFs) provide noisy votes that are aggregated into silver labels. Each LF $\lambda_j : w_i \to \{0, 1, \perp\}$ examines a window and emits a vote: stagnation (0), progress (1), or abstain ($\perp$).

3.2.1 Labeling Functions

We design three LFs that target complementary signals of stagnation.

Command repetition (λ_1). Computes the proportion of repeated commands within w_i; a high ratio triggers a stagnation vote. Never abstains. Fires on 65.5% of SWE-Agent windows but only 5.4% of OH windows (97.9% precision), reflecting agent-specific temporal dynamics (§4.6).

File diff delta (λ_2). Measures the fraction of steps producing code modifications. Low modification frequency triggers stagnation; >1 edited file triggers progress; otherwise abstains. On OH, achieves 87.5% recall (99.9% precision), the most reliable cross-coding-agent signal.

Outcome backpropagation (λ_3). Uses the final task outcome as distant supervision: unresolved tasks' last windows receive stagnation votes; resolved tasks receive progress votes; all others abstain. Unlike λ_1/λ_2, λ_3 propagates a global signal backward through time.

3.2.2 Label Aggregation

Silver labels are produced by majority vote among non-abstaining LFs:

$$y_i = \text{MV}\big(\{\lambda_j(w_i) : \lambda_j(w_i) \neq \perp\}\big) \quad (1)$$

We define the confidence of each label as the number of non-abstaining LFs that agree with the winning label: $\text{conf}_i = |\{j : \lambda_j(w_i) \neq$

$\perp$ and $\lambda_j(w_i) = y_i\} | \in \{1, 2, 3\}$. Conf=1 arises when a single non-abstaining LF determines the label—either because only one LF votes or because two LFs disagree with no clear majority—producing the most ambiguous labels. The conf$\geq$2 subset, where at least two non-abstaining LFs agree on the winning label, serves as the primary evaluation set throughout this work. This stratification allows us to separately analyze model behavior on high-confidence vs. ambiguous windows (§4.2).

3.3 Detection Methods

3.3.1 Structural Features

We extract a 30-dimensional feature vector from each window, requiring no pretrained model or GPU. Features span four categories: (1) *repetition*: exact command repeat ratio, fuzzy match ratio (Levenshtein-based), and unique command ratio; (2) *file operations*: counts of file creates, modifications, and deletions within the window; (3) *edit patterns*: edit distances between consecutive actions, capturing how much the agent's behavior changes step-to-step; (4) *output statistics*: entropy of command outputs and action type distribution features. These features are fed to three classifiers: Logistic Regression (LR), Random Forest (RF), and Gradient Boosting (GB). Because several structural features overlap with the signals used by the labeling functions (e.g., repetition ratios relate to λ_1), structural classifiers can partially reconstruct the LF voting process. We discuss this circularity and its implications in §4.2.

3.3.2 NL Embedding Probe (ProgressProbe)

Each action a_t is encoded as a natural language description and embedded using Qwen3-Embedding-0.6B (Zhang et al., 2025) into a 1024-dimensional vector. The k embeddings within a window form a sequence that is processed by a 2-layer GRU (Cho et al., 2014) with hidden size 256 and dropout 0.1. An attention mechanism (Bahdanau et al., 2015) computes a weighted combination of the GRU hidden states, yielding a single window-level representation. This representation is passed through a fully connected layer with sigmoid activation for binary classification. Training uses binary cross-entropy loss with early stopping (patience 5) on a validation set.

The GRU architecture is motivated by the sequential nature of actions within a window: the order in which commands are issued (e.g., repeated failures followed by a different approach) carries diagnostic information that bag-of-embeddings methods discard. We also evaluate two simpler embedding baselines to isolate the contribution of temporal modeling: (1) mean-pooled Qwen3 embeddings with logistic regression, and (2) cosine similarity between consecutive window embeddings with a validation-tuned threshold.

3.3.3 LLM-as-Judge Baseline

We prompt Claude Sonnet 4.6 in zero-shot mode to classify each window as stagnation or progress, given the textual descriptions of actions within the window.[1] The LLM judge establishes a cost-performance reference: at $11.66 per test set, its zero-shot F1 of 0.663 demonstrates that stagnation detection is non-trivial even for capable LLMs without task-specific adaptation. This baseline is not used in the silver label pipeline.

3.4 Temporal Label Models

The three LFs occupy distinct temporal niches relative to stagnation onset (§4.5): λ_1 leads by $\sim$2 steps, λ_2 is near-concurrent, and λ_3 lags by $\sim$7 steps. This motivates testing whether temporal aggregation improves over majority vote. We evaluate three temporal label models—Time-Varying Weights (TWV) with Gaussian kernel smoothing, a Hidden Markov Model (HMM) (Safranchik et al., 2020) over joint LF patterns, and a Lag-Aware HMM that models each LF's temporal offset—alongside two generative i.i.d. models: FlyingSquid (Fu et al., 2020) and Snorkel MeTaL (Ratner et al., 2019b)—all descendants of classical EM-based annotator models (Dawid and Skene, 1979). Full model specifications appear in Appendix A.8.

4 Experiments

We organize results around three questions: (1) which method best detects stagnation across deployment scenarios? (§4.2–§4.4); (2) what drives label quality and cross-coding-agent transfer bottlenecks? (§4.5–§4.6); (3) can temporal structure in LFs improve labels? (§4.7). Silver-label metrics (Tables 1–2) reflect LF-agreement scores; the gold set (§4.8) provides non-circular stagnation detection accuracy.

Circular evaluation caveat. Structural features overlap with the signals LFs use to produce silver labels; hence in-domain structural F1 (0.995)

[1] Extended LLM evaluation including chain-of-thought prompting, few-shot examples, and multi-model comparison is reported in Appendix A.7.

reflects LF reverse-engineering, not ground-truth accuracy. Cross-agent evaluation on OpenHands (0.967) and the gold set (§4.8) provide non-circular evidence (§5.1).

4.1 Experimental Setup

Datasets. Sliding windows of k=5 consecutive actions are extracted from SWE-Agent (54,809 windows, 28.5% stagnation) (Jimenez et al., 2024; Yang et al., 2024a) and OpenHands (150,652 windows, 20.9% stagnation) (Wang et al., 2025a) trajectories on SWE-bench. Silver labels are produced by majority vote; confidence equals the count of agreeing LFs (1–3). Splits are at the trajectory level (70/15/15%); evaluation uses conf≥2: N=1,824 (SWE-Agent) and N=43,376 (OH).

Detection methods. (1) **Structural features**: 30 hand-crafted features (repetition rates, file operation counts, edit distances, output entropy) fed to Logistic Regression (LR), Random Forest (RF), and Gradient Boosting (GB). (2) **ProgressProbe**: Qwen3-Embedding-0.6B (Zhang et al., 2025) embeddings (1024-dim) processed by a 2-layer GRU (Cho et al., 2014) with attention (Bahdanau et al., 2015) (hidden=256, dropout=0.1), early-stopped at epoch 3 with patience 5. (3) **Mean-pool + LR**: Mean-pooled Qwen3 embeddings with logistic regression. (4) **LLM-as-Judge**: Claude Sonnet 4.6 zero-shot window-level classification ($11.66 API cost for the SWE test set). (5) **Cosine similarity**: Cosine distance between consecutive window embeddings with a validation-tuned threshold. (6) **N-gram cycle detector**: Flags stagnation when an exact n-gram ($n \in \{2, 3\}$) of commands recurs within a window—a simple repetition baseline absent from our initial comparison.

Metrics. Macro-F1 is the primary metric. We also report per-class F1 for stagnation ($F1_{stag}$) and progress ($F1_{prog}$).

4.2 Main Results

Table 1 presents in-domain results on the SWE-Agent conf≥2 test set and cross-coding-agent transfer to OpenHands.

In-domain, structural RF achieves 0.995 macro-F1, exceeding the GRU probe by 8.96 pp (paired bootstrap: 95% CI [7.44, 10.57] pp, p<0.001). However, the structural–probe gap is inflated by circular evaluation: structural features reconstruct the LFs that generated the labels. The more informative comparison is among non-circular methods: the GRU probe outperforms mean-pool + LR by 6.2 pp, confirming that temporal modeling within windows adds value beyond static embedding aggregation. The zero-shot LLM judge (0.663) underperforms all learned methods despite higher cost ($11.66 per test set). Extended evaluation (Appendix A.7) yields a best LLM macro-F1 of 0.799 on silver (five-shot-CoT); on gold, the ranking reverses: LLM 0.910 surpasses structural GB 0.839 (§4.8), illustrating silver-label circularity. CoT and few-shot yield comparable gains (+8.7 vs. +8.9 pp); combining them adds +4.8 pp.

4.3 Cross-Agent Transfer

Table 1 (right columns) shows models trained on SWE-Agent and evaluated on OpenHands without any OH training data. Structural GB transfers best, achieving 0.967 macro-F1 on all OH windows. RF drops to 0.912 on all-confidence OH data despite matching GB in-domain, suggesting GB's boosted ensemble is more robust to distributional shift. Both structural models exceed 0.99 macro-F1 on conf≥2 OH windows (GB 0.992, RF 0.996), where label noise is lower. Transfer is symmetric: OH→SWE achieves 0.995 for both GB and RF on conf≥2, comparable to SWE→OH (0.992/0.996), confirming agent-agnostic structural patterns (Appendix Table 16). The LLM judge degrades further on OH (0.491 zero-shot; N=500), 47.6 pp below structural GB—few-shot-CoT (0.470) underperforms zero-shot, as SWE-Agent examples induce negative transfer.

On the same OH conf≥2 test split (N=6,122), the probe's zero-shot transfer (0.598) lags structural GB (0.993) by 39.4 pp (95% CI [37.9, 41.0]), but full OH fine-tuning closes this gap: the fine-tuned probe reaches 0.976, surpassing GB (0.967) on all windows (§4.6). The practical implication is a deployment trade-off: structural features when no labeled data is available; probe fine-tuning when annotation budget permits. This crossover is scope-dependent: on conf≥2, GB retains its advantage (0.992 vs. 0.976; p<0.001), suggesting the probe's all-windows edge reflects tolerance of noisy conf=1 labels rather than superior detection.

Multi-seed stability. Structural RF: 0.995±0.000 in-domain (3 seeds); GB cross-coding-agent: 0.968±0.002. The probe shows higher variance (0.919±0.011) but all seeds exceed 0.90.

Method	SWE-Agent In-Domain (conf≥2)			OpenHands Transfer	
	Macro-F1	$F1_{stag}$	$F1_{prog}$	All	Conf≥2
Structural RF†	**0.9951**	0.9928	0.9974	0.9123	**0.9959**
Structural GB†	**0.9951**	0.9928	0.9974	**0.9674**	0.9918
Structural LR†	0.9874	0.9815	0.9933	0.7801	0.9671
ProgressProbe (GRU)	0.9055	0.8593	0.9518	0.530[a]	0.598[a]
+ full OH fine-tune	—	—	—	0.976[b]	0.976[b]
Mean-pool + LR	0.8435	0.7620	0.9250	—	—
LLM-as-Judge	0.6625	0.6055	0.7195	—	—
Cosine Similarity	0.5606	—	—	0.447	0.393

Table 1: Stagnation detection results (SWE-Agent in-domain, N=1,824; OpenHands zero-shot transfer). †Structural metrics reflect LF-reconstructibility, not ground-truth accuracy (§5.1). [a]Probe All: full OH embeddings (N=143,057); Conf≥2: 15% test split (N=6,122). Structural models on all OH windows (N=150,652) and conf≥2 (N=43,376). [b]Paired bootstrap (N=6,122): GB leads fine-tuned probe by 1.65 pp (CI [1.11, 2.23], p<0.001) on conf≥2; probe advantage on all windows reflects better conf=1 performance. **Bold** = best, underline = second.

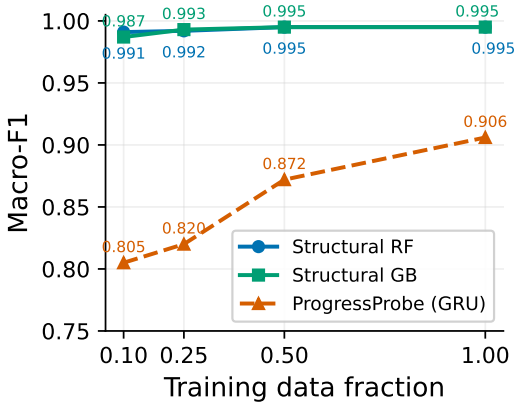


Figure 2: Macro-F1 vs. training data fraction. Structural classifiers saturate by 50%; the probe has not plateaued at 100%.

LF Config	Macro-F1	$F1_{stag}$	$F1_{prog}$
3-LF (w/ cmd_rep)	**0.9763**	0.9600	0.9930
2-LF (no cmd_rep)	0.6822	0.4810	0.8840
Δ	−29.4 pp	−47.9 pp	−10.9 pp

Table 2: Prevalence-matched ablation on the same OH test set (N=6,122, stag rate 19.3%). Removing cmd_rep drops macro-F1 by 29.4 pp (McNemar χ^2=1551, p≈0).

4.4 Robustness Analysis

In-domain performance is encoder-agnostic (0.93–0.94 across three encoders; Appendix Table 3), and a Transformer encoder is equivalent to GRU (+0.18 pp, p=0.82). Cross-agent transfer varies with encoder choice (0.60–0.73, 13.5 pp spread), but even the best probe (E5-large, 0.733) remains 25.9 pp below structural GB (0.992), confirming the structural advantage is robust across encoders (Appendix Table 15). Window size k=5 is optimal; k=3 drops 5.9 pp, k=7 drops 1.8 pp (Appendix Table 4). Structural classifiers are extremely data-efficient: RF achieves 0.991 and GB 0.987 at just 10% training data, fully saturating by 50%—a 20× advantage over the probe (Figure 2).

4.5 Labeling Function Temporal Dynamics

The three LFs occupy distinct temporal niches (Table 11, Figure 3 in Appendix A.15): cmd_rep leads by 1.87 steps, file_diff is near-concurrent (−1.20), and outcome_bp lags by 6.70 steps (all pairwise p<10^{-26}). On OpenHands, cmd_rep reverses to lagging (+3.32 steps), firing on only 5.4% of windows with 97.9% precision—a fundamentally different temporal role than on SWE-Agent.

4.6 Label Quality and cmd_rep Coverage

A label quality audit revealed a version mismatch in OH embedding vectors; after correction, the OH stagnation rate shifted from 6.56% to 21.4%, and 5% fine-tuning F1 improved from 0.570 to 0.850—demonstrating that label quality, not model capacity, was the transfer bottleneck (full curve in Appendix Table 6).

Removing cmd_rep drops macro-F1 by 29.4 pp (Table 2; McNemar p≈0) due to data volume collapse: cmd_rep expands training data ∼10× (4K→43K windows) while agreeing 100% with 2-LF labels on overlapping windows—purely a coverage contribution (Appendix A.9).

4.7 Temporal Label Models

Five label models—three temporal (TWV, HMM, Lag-HMM) and two generative (FlyingSquid,

Snorkel MeTaL)—all fail to improve over majority vote (Table 10 in Appendix A.10); the best (TWV, +0.6 pp self-evaluated) degrades downstream probe F1 by 2.57 pp. The bottleneck is LF quantity: three binary LFs produce at most $2^3{=}8$ voting patterns with insufficient inter-LF conflict to exploit. A 6-LF expansion confirms this: HMM recovers +6.5 pp but remains below 3-LF MV in absolute terms (Appendix A.21).

4.8 Gold Set Validation

Three CS graduate students annotate a stratified gold set ($N{=}505$, ≥ 2 annotators agree, stagnation prevalence 26.3% vs. silver 24.6%; silver–gold agreement 93.8%, $\kappa{=}0.865$; protocol in Appendix A.16). GB achieves 0.839 gold macro-F1 vs. 0.798 for MV (+4.2 pp, bootstrap $p{<}0.001$, 95% CI [0.758, 0.837]); probe models use the SWE-only subset ($N{\approx}335$, Table 12).

LLM gold evaluation. The best LLM (few-shot-CoT) achieves 0.910 gold macro-F1, surpassing GB (0.839) by 7.1 pp—reversing the silver ranking—but LLM judges collapse on cross-agent transfer (best: 0.491 vs. 0.967, −47.6 pp; §4.3).

Cycle detector comparison. N-gram cycle detectors match structural GB on in-domain gold (∼0.84) but collapse cross-agent (−43.7 pp; Appendix Table 14).

5 Analysis

5.1 Circular Evaluation and Its Implications

The structural RF's 0.995 macro-F1 (§4.2) is a predictable consequence of feature–label overlap: the 30 features include repetition ratios and file operation counts—the same signals λ_1 and λ_2 consume to produce silver labels—so this score represents an upper bound on LF reverse-engineering fidelity (§5.2 quantifies this decomposition).

Three settings break this circularity: (1) cross-coding-agent transfer (GB 0.967 on OH, where cmd_rep fires on 5.4% vs. 65.5% of windows—LF memorization cannot explain this); (2) instance isolation (removing 284 shared SWE-bench tasks changes cross-agent F1 by <0.1 pp, Table 8); (3) gold validation (GB 0.839 vs. MV 0.798, $p{<}0.001$; §4.8). In-domain structural metrics should still be interpreted as LF-agreement scores. The LLM judge illustrates this danger: on silver, structural features lead the best LLM by 19.6 pp (0.995 vs. 0.799), but on gold the ranking reverses (LLM 0.910 vs. GB 0.839)—the silver advantage was an artifact of LF-reconstructibility.

5.2 Feature Overlap Ablation

We partition the 30 features into 11 *LF-overlapping* and 19 *residual* (Appendix A.17). On all OH windows, 11 LF-overlapping features alone achieve 0.977 cross-agent F1, exceeding the full 30-feature model by 1.0 pp—the opposite of what leakage would predict. However, this reverses on conf$\geq$2: 11-feature F1 drops to 0.985 vs. 0.992 for 30 features (−0.7 pp), indicating that residual features provide incremental value for noise filtering on high-confidence labels. Residual features collapse to 0.566 cross-agent (0.952 in-domain), confirming agent-specific noise. We acknowledge that cross-agent transfer primarily validates the transferability of LF-encoded judgment patterns across agents, rather than demonstrating that models learn signals beyond the LFs. Leave-one-group-out localizes the signal: removing cmd_rep drops cross-agent F1 by 40.6 pp; file_diff by 30.8 pp; outcome_bp by only 0.2 pp. In resource-constrained settings, the 11 LF-overlapping features suffice; for high-confidence deployment, the full 30-feature set is preferred.

GB vs. RF deployment trade-off. GB leads RF by 5.5 pp on all-windows cross-agent (0.967 vs. 0.912) but RF is more robust on conf$\geq$2 and under 2-LF degradation (Appendix A.14); either enables 15–30% compute savings (Appendix Figure 7).

6 Conclusion

We compared structural features, neural probes, LLM judges, and cycle detectors for stagnation detection in LLM coding agents. On gold, the best LLM judge achieves the highest non-circular accuracy (0.910 vs. structural GB 0.839), but LLM judges collapse on cross-agent transfer (0.491 vs. 0.967, −47.6 pp); cycle detectors likewise collapse (−43.7 pp). Structural features uniquely enable zero-data transfer (+39.4 pp over probe); with fine-tuning, the probe surpasses structural features (0.976 vs. 0.967). Five label models fail to improve over majority vote; future work includes non-SE domains, online integration, and scaling to >10 LFs. Code and trained models will be released upon acceptance.

Limitations

Scope. We evaluate two software engineering coding agents (SWE-Agent and OpenHands) on

SWE-bench. Cross-coding-agent transfer is demonstrated between these two agents. A pilot on ALFWorld (Appendix A.23) suggests 23/30 features transfer to non-SE domains, but full evaluation with domain-specific gold labels remains future work.

Third-agent transfer. Extending the pipeline to a third coding agent (ACR; Appendix A.22) reveals that LF portability—not model architecture—is the transfer bottleneck: 2 of 4 adapted LFs achieve zero coverage, and adapted LFs yield inconsistent gains across classifiers (RF $+10.4$ pp, GB -3.6 pp). Each new agent likely requires agent-specific LF design.

Silver labels. Silver-label evaluation dominates this study; while a 505-window gold set provides partial non-circular validation (§4.8), the primary metrics in Tables 1–5 remain silver-evaluated, and the structural RF's 0.995 macro-F1 reflects LF-reconstructibility rather than ground-truth detection accuracy. Moderate inter-annotator agreement (Fleiss' κ=0.39) reflects the inherent subjectivity of stagnation boundaries.

Labeling function count. Three LFs produce insufficient inter-LF conflict (95.1% agreement at conf$\geq$2) for any label model—temporal or generative—to demonstrate value; scaling to $>$10 LFs with higher disagreement is needed.

No online evaluation. All results are offline; no intervention or real-time deployment evaluation has been conducted.

Label version sensitivity. Cross-coding-agent transfer results are sensitive to label quality: a single embedding version mismatch shifted the OH stagnation rate from 6.56% to 21.4% and improved 5% fine-tuning F1 from 0.570 to 0.850. This sensitivity suggests that deploying the pipeline on new agents requires careful label auditing before conclusions about transfer difficulty can be drawn.

Outcome dependence. The outcome backpropagation labeling function (λ_3) requires task outcome, which is unavailable during inference.

Code and data. Code, data, and trained models are anonymous during review and will be released upon acceptance.

References

Ryan Prescott Adams and David J.C. MacKay. 2007. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*.

Samaneh Aminikhanghahi and Diane J. Cook. 2017. A survey of methods for time series change point detection. *Knowledge and Information Systems*, 51(2):339–367.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *International Conference on Machine Learning*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, and 1 others. 2025. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*.

Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3).

Yueh-Han Chen, Nitish Joshi, Yulin Chen, He He, and Rico Angell. 2025. Monitoring llm agents for sequentially contextual harm. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, and 1 others. 2014. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing*.

Sanjiban Choudhury. 2025. Process reward models for llm agents: Practical framework and directions. *arXiv preprint arXiv:2502.10325*.

A. P. Dawid and A. M. Skene. 1979. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Applied Statistics*, 28(1):20–28.

Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *ACM SIGSAC Conference on Computer and Communications Security*.

Kaixuan Fan, Kaituo Feng, Manyuan Zhang, and 1 others. 2026. Exploring reasoning reward model for agents. *arXiv preprint arXiv:2601.22154*.

Daniel Y. Fu, Mayee F. Chen, Frederic Sala, and 1 others. 2020. Fast and three-rious: Speeding up weak supervision with triplet methods. In *International Conference on Machine Learning*.

Felix George, Harshit Kumar, Divya Pathak, and 1 others. 2025. Unsupervised cycle detection in agentic applications. *arXiv preprint arXiv:2511.10650*.

Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor W. Tsang, and Masashi Sugiyama. 2018. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems*.

Jie Huang, Xinyun Chen, Swaroop Mishra, and 1 others. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Carlos E. Jimenez, John Yang, Alexander Wettig, and 1 others. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*.

Neil Kale, Chen Bo Calvin Zhang, Kevin Zhu, and 1 others. 2025. Reliable weak-to-strong monitoring of llm agents. *arXiv preprint arXiv:2508.19461*.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics*.

Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4).

Martin Leucker and Christian Schallhart. 2009. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303.

Dawei Li, Yuguang Yao, Zhen Tan, Huan Liu, and Ruocheng Guo. 2026. Toolprmbench: Evaluating and advancing process reward models for tool-using agents. *arXiv preprint arXiv:2601.12294*.

Junnan Li, Richard Socher, and Steven C.H. Hoi. 2020. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *IEEE International Conference on Data Mining*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*.

Yibing Liu, Chong Zhang, Zhongyi Han, and 1 others. 2026. Trajad: Trajectory anomaly detection for trustworthy llm agents. *arXiv preprint arXiv:2602.06443*.

David Manheim and Scott Garrabrant. 2018. Categorizing variants of goodhart's law. *arXiv preprint arXiv:1803.04585*.

Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *International Joint Conference on Artificial Intelligence*.

Gabriel Orlanski, Devjeet Roy, Alexander Yun, and 1 others. 2026. Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*.

Charilaos Pipis, Shivam Garg, Vasilis Kontonis, and 1 others. 2025. Wait, wait, wait... why do reasoning models loop? *arXiv preprint arXiv:2512.12895*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*.

Rath and 1 others. 2026. Agent drift: Quantifying behavioral degradation in multi-agent LLM systems. *arXiv preprint*.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, and 1 others. 2019a. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*.

Alexander Ratner, Braden Hancock, Jared Dunnmon, and 1 others. 2019b. Training complex models with multi-task weak supervision. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Alexander Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Re. 2016. Data programming: Creating large training sets, quickly. In *Advances in Neural Information Processing Systems*.

Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.

Esteban Safranchik, Shiying Luo, and Stephen H. Bach. 2020. Weakly supervised sequence tagging from noisy rules. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. 2020. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *International Conference on Pattern Recognition*.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Liu, Shuo Liu, Zhiyuan Liu, and Maosong Sun. 2024. Debugbench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Wil M. P. van der Aalst. 2016. *Process Mining: Data Science in Action*, 2nd edition. Springer.

Paroma Varma, Frederic Sala, Shiori Sagawa, Jason Fries, Daniel Y. Fu, Saelig Khattar, Ashwini Ramamoorthy, Ke Xiao, Kayvon Fatahalian, James Priest, and Christopher Ré. 2019. Multi-resolution weak supervision for sequential data. In *Advances in Neural Information Processing Systems*.

Jiaxuan Wang, Yulan Hu, Wenjin Yang, and 1 others. 2026. Aligning agents via planning: A benchmark for trajectory-level reward modeling. *arXiv preprint arXiv:2604.08178*.

Xingyao Wang, Boxuan Li, Yufan Song, and 1 others. 2025a. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*.

Zijian Wang and 1 others. 2025b. Probguard: Probabilistic runtime monitoring for LLM agent safety. *arXiv preprint arXiv:2508.00500*.

Zhiheng Xi, Chenyang Liao, Guanyu Li, and 1 others. 2025. Agentprm: Process reward models for llm agents via step-wise promise and progress. *arXiv preprint arXiv:2511.08325*.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2024. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*.

John Yang, Carlos E. Jimenez, Alexander Wettig, and 1 others. 2024a. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2024b. Intercode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, and 1 others. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Eric Zhan, Stephan Zheng, Yisong Yue, Long Sha, and Patrick Lucey. 2019. Generating multi-agent trajectories using programmatic weak supervision. In *International Conference on Learning Representations*.

Jieyu Zhang, Yue Yu, Yinghao Li, and 1 others. 2021. Wrench: A comprehensive benchmark for weak supervision. In *Advances in Neural Information Processing Systems*.

Yanzhao Zhang, Mingxin Li, Dingkun Long, and 1 others. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024a. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024b. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, and 1 others. 2025. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

A Supplementary Results

A.1 Encoder Ablation

A.2 Window Size Ablation

A.3 Multi-Seed Stability

A.4 Cross-Agent Label Quality and Few-Shot Transfer

Table 6 extends the label quality analysis from §4.6, showing probe fine-tuning performance across data fractions under corrected OH labels.

The probe reaches 0.948 macro-F1 at 20% OH data—within 5 pp of structural GB on conf$\geq$2 windows (0.992)—and at 50% (0.968) closes to within

Encoder	Dim	Macro-F1	$F1_{stag}$	$F1_{prog}$
Qwen3-0.6B	1024	0.9055	—	—
E5-large-v2	1024	0.9405 ± 0.0030	—	—
BGE-M3	1024	**0.9413**	**0.9147**	**0.9679**

Table 3: Encoder ablation on SWE-Agent conf$\geq$2 (N=1,824). All models use the same GRU architecture (2-layer, hidden=256, dropout=0.1, lr=1e-3, batch=32, early stopping patience 5). All encoders exceed 0.90 macro-F1, confirming the probe's temporal architecture drives performance rather than a specific encoder. BGE-M3 achieves the highest score (+3.6 pp over Qwen3), but the gap between encoders (0.5 pp) is small relative to the structural–probe gap (8.96 pp). "—" indicates per-class scores were not logged in the initial sweep; macro-F1 is computed from the checkpoint with lowest validation loss. E5-large-v2 reports mean $\pm$ std over 3 seeds.

Window k	Macro-F1	$F1_{stag}$	$F1_{prog}$	Δ vs. k=5
3	0.8721	0.7784	0.9659	−5.9 pp
5	**0.9309**	**0.8917**	**0.9701**	—
7	0.9133	0.8791	0.9475	−1.8 pp

Table 4: Window size ablation on SWE-Agent conf$\geq$2 using the Qwen3-0.6B encoder (seed 42). k=5 is optimal. k=3 lacks sufficient context for stagnation detection ($F1_{stag}$ drops 11.3 pp). k=7 dilutes signal by spanning both progress and stagnation phases.

2.5 pp of structural GB on conf$\geq$2 (0.992). This rapid convergence under corrected labels contrasts sharply with the old label regime, where even 100% fine-tuning reached only 0.751. The improvement is driven entirely by label quality: the model architecture and training procedure are identical.

Under the prevalence-matched ablation (§4.6), 3-LF and 2-LF label sets agree on 100% of overlapping windows, confirming that cmd_rep's contribution is coverage (expanding conf$\geq$2 training data from $\sim$4K to $\sim$43K windows) rather than label refinement. The -29.4 pp drop upon removing cmd_rep (McNemar χ^2=1,551, $p\approx$0) is thus attributable to data volume collapse, not label noise.

A.5 Structural Feature Definitions

Table 7 lists all 30 structural features with their categories and LF overlap classification. A feature is classified as LF-overlapping if it is computed from the same signal space that any labeling function uses as input (§5.2).

Method	In-Domain F1	Seeds	Cross-Agent F1	Se
Structural RF	0.9951 ± 0.0000	5	—	–
Structural GB	—	—	0.9678 ± 0.0015	5
ProgressProbe (GRU)	0.9185 ± 0.0113	5	—	–

Table 5: Multi-seed stability (mean $\pm$ std). In-domain: SWE-Agent conf$\geq$2; cross-coding-agent: SWE-Agent $\rightarrow$ OpenHands zero-shot transfer. Structural classifiers show zero or near-zero variance. The probe exhibits higher variance (σ=0.011) but all seeds exceed 0.90. "—" indicates the setting was not evaluated: RF is evaluated in-domain only, GB cross-coding-agent only, and the probe in-domain only.

OH Data Fraction	Probe Macro-F1
0% (zero-shot)	0.598
5%	0.850
10%	0.918
20%	0.948
50%	0.968
100%	0.976

Table 6: Probe fine-tuning on corrected OH labels across data fractions. The zero-shot–to–5% jump (+25.2 pp) and rapid convergence toward structural GB (0.992 on conf$\geq$2) demonstrate that label quality, not model capacity, was the transfer bottleneck. Under old (corrupted) labels, 5% fine-tuning reached only 0.570.

A.6 Instance Isolation

A.7 Extended LLM-as-Judge Evaluation

Table 9 extends the zero-shot LLM-as-judge baseline (Table 1) with chain-of-thought (CoT) prompting, few-shot examples, and an alternative model, all evaluated on the SWE-Agent conf$\geq$2 test set (N=1,824).

A.8 Temporal Label Model Specifications

A.8.1 Time-Varying Weights (TWV)

TWV applies Gaussian kernel smoothing to each LF's activation sequence:

$$\tilde{\lambda}_j(t) = \sum_{t'} K_\sigma(t - t') \cdot \lambda_j(t') \qquad (2)$$

where K_σ is a Gaussian kernel with bandwidth σ. Smoothed activations replace raw votes for conf=1 windows (where exactly one LF disagrees), potentially flipping labels in temporally ambiguous regions. The bandwidth σ controls the extent of temporal context: larger values incorporate more distant steps.

A.8.2 Hidden Markov Models

We model the label sequence as a Hidden Markov Model (Safranchik et al., 2020) with two hid-

#	Feature	Category	LF Overlap
1	num_steps	Trajectory stats	—
2	num_actions	Trajectory stats	—
3	num_observations	Trajectory stats	—
4	action_ratio	Trajectory stats	—
5	mean_text_len	Text statistics	—
6	std_text_len	Text statistics	—
7	max_text_len	Text statistics	—
8	min_text_len	Text statistics	—
9	mean_action_len	Text statistics	—
10	mean_obs_len	Text statistics	—
11	text_len_trend	Text statistics	—
12	num_commands	Command stats	—
13	mean_cmd_len	Command stats	—
14	max_cmd_len	Command stats	—
15	unique_cmd_ratio	Repetition	λ_1
16	max_repeat_ratio	Repetition	λ_1
17	num_edit_cmds	File operations	λ_2
18	num_search_cmds	File operations	λ_2
19	num_nav_cmds	File operations	—
20	num_submit_cmds	File operations	—
21	edit_ratio	File operations	λ_2
22	search_ratio	File operations	λ_2
23	avg_adjacent_cmd_sim	Edit distance	λ_1
24	max_adjacent_cmd_sim	Edit distance	λ_1
25	max_consec_similar	Edit distance	λ_1
26	num_errors	Output statistics	—
27	num_test_outputs	Output statistics	—
28	num_file_displays	Output statistics	λ_2
29	num_nonzero_exits	Output statistics	—
30	is_short_trajectory	Trajectory stats	λ_3

Table 7: Complete structural feature definitions (N=30). LF Overlap indicates which labeling function's signal space the feature shares: λ_1 (command repetition), λ_2 (file diff), λ_3 (outcome backpropagation), or "—" (no overlap). 11 features overlap with at least one LF; 19 are residual.

Model	Full	Unseen-only	Δ
GB all	0.9674	0.9669	−0.05 pp
RF all	0.9123	0.9121	−0.01 pp
GB conf≥2	0.9919	0.9917	−0.02 pp
RF conf≥2	0.9959	0.9962	+0.03 pp

Table 8: Instance isolation: cross-coding-agent macro-F1 with and without 284 SWE-bench task instances that overlap between the 1,386 SWE-bench training instances and 3,409 OH test instances (8.3% of OH). All $|\Delta| < 0.1$ pp, confirming transfer reflects behavioral pattern generalization rather than instance memorization.

den states (stagnation, progress) and observations defined by the joint activation pattern of the three LFs. Transition and emission parameters are estimated via Expectation-Maximization (EM). The HMM captures temporal persistence—stagnation episodes tend to span multiple consecutive windows—which majority vote, operating independently per window, cannot model.

A.8.3 Lag-Aware HMM

The standard HMM treats all LFs as synchronous. Lag-Aware HMM extends this by explicitly modeling each LF's temporal offset: λ_1 is shifted by -2 steps, λ_2 by -1, and λ_3 by $+7$, estimated from the temporal dynamics analysis (§4.5). To prevent divergence, we apply two regularization strategies: (1) *confidence anchoring*, which fixes labels for conf≥2 windows and restricts EM updates to conf=1 windows; and (2) a prior strength of 5000 on transition parameters. These constraints keep the model close to the majority vote baseline while allowing it to resolve temporally ambiguous labels.

A.9 Label Quality: Extended Analysis

With corrected labels, the 3-LF probe improves from zero-shot 0.598 to fine-tuned 0.976 (previously 0.7512 under corrupted labels). The improvement is most dramatic at low data fractions: 5% fine-tuning reaches 0.850, exceeding the old 50% result (0.69) by 16.0 pp. The probe converges rapidly: at 50% OH data it reaches 0.968, essentially matching structural GB (0.967).

Model	Setting	Macro-F1	$F1_{stag}$	Cost
Claude Sonnet 4.6	zero-shot	0.6625	0.6055	$11.66
Claude Sonnet 4.6	CoT	0.7499	0.6808	$17.16
Claude Sonnet 4.6	5-shot	0.7514	0.6828	$36.08
Claude Sonnet 4.6	5-shot-CoT	**0.7994**	**0.7306**	$40.56
GPT-5.5[†]	zero-shot	0.7465	0.7110	$11.62
Structural feature augmentation				
Claude Sonnet 4.6	+ structural input[‡]	0.6880	0.6270	$14.82

Table 9: Extended LLM-as-judge evaluation on SWE-Agent conf≥2 (N=1,824). All Claude rows use the same model; the performance gain comes from prompting strategy. CoT alone yields gains comparable to few-shot (+8.7 pp vs. +8.9 pp over zero-shot); combining them (five-shot-CoT) adds a further 4.8 pp, suggesting complementary mechanisms. [‡]30 structural features provided as formatted text in the LLM prompt; performance degrades below pure few-shot (0.7514), indicating structural signals require engineered classifiers. [†]GPT-5.5: 22.4% of responses (408/1,824) failed to parse; macro-F1 is computed over the 1,416 windows with valid outputs. On silver, the best LLM (0.799) trails structural features (0.995); on gold, the ranking reverses (LLM 0.910 vs. GB 0.839; §4.8). Total API cost across all configurations: $131.90.

Method	MV Agr.	Conf≥2 F1	Δ vs MV	Probe F1[‡]
MV (baseline)	1.000	0.8946[*]	—	0.8468
Temporal				
TWV (σ=2)	0.885	**0.9003**	+0.6 pp	0.8211
Lag-HMM	0.820	0.8936	−0.1 pp	—
HMM	0.685	0.7502	−14.4 pp	0.6564
Generative (i.i.d.)				
FlyingSquid	0.820	0.7820	−11.3 pp	—
Snorkel MeTaL	0.423	0.2580	−63.6 pp	—

Table 10: Label model comparison. Conf≥2 F1: each model's labels evaluated against MV labels on conf≥2 windows. [*]MV conf≥2 F1 (0.8946) reflects downstream probe performance when trained on MV labels, not self-agreement (which would be 1.0). [‡]Downstream probe trained on each method's full labels, evaluated on the SWE-Agent test set (N=8,101) against MV labels. "—" = not evaluated downstream. No model consistently improves over MV.

The 29.4 pp drop upon removing cmd_rep is not caused by label accuracy improvement. After correction, cmd_rep fires on only 5.39% of OH windows (down from the previously reported 92.2% under corrupted embeddings) with 97.91% precision—a high-precision, low-recall signal. On the 4,070 windows where both 2-LF and 3-LF labels are defined, the two configurations agree on 100% of labels: cmd_rep does not change any label where the other two LFs already vote. Its contribution is purely *coverage*: including cmd_rep expands the training set from ∼4K to ∼43K windows (a ∼10× increase). The 2-LF model's low F1 reflects data volume collapse—too few training examples to learn the stagnation distribution—rather than label corruption.

These results reframe how labeling functions contribute to weak supervision pipelines. A high-precision, low-recall LF like cmd_rep may appear redundant when its labels are a subset of the majority vote, yet removing it causes training data to collapse by an order of magnitude. The core contribution of such LFs is coverage provision, not label refinement.

A.10 Temporal Label Model Results

Temporal models. TWV achieves the highest conf≥2 F1 (0.900, +0.6 pp over MV), but this gain does not propagate downstream: a probe trained on TWV labels scores 0.821 vs. 0.847 for MV-trained (−2.57 pp). TWV flips 14.1% of conf=1 labels, introducing noise that hurts probe training. The Gaussian kernel bandwidth σ is critical: σ=2 is optimal; σ=1 drops to 0.811 and σ=5 to 0.808, making TWV impractical as a drop-in MV replacement. HMM overpredicts stagnation (29,176 vs. 25,633 progress windows), degrading conf≥2 F1 by 14.4 pp and downstream performance to 0.656. Lag-HMM closely tracks MV (−0.1 pp) despite explicitly modeling temporal offsets.

Generative (i.i.d.) models. FlyingSquid (Fu et al., 2020), a closed-form triplet-based model, drops 11.3 pp below MV. On the full dataset, agreement with MV is 82%; all divergent labels flip progress→stagnation, introducing systematic stagnation bias. Snorkel MeTaL (Ratner et al., 2019b) fails catastrophically (−63.6 pp), with only 42.3% MV agreement. Both models assume conditional independence among LFs—an assumption violated by the shared behavioral signals underlying our three LFs—and lack the sequential structure to recover from this misspecification.

Root cause: insufficient inter-LF conflict. Three binary LFs produce at most 2^3=8 voting patterns; conf≥2 windows dominate the training set, leaving insufficient ambiguity for any aggregation method to exploit. In the current 3-LF, low-conflict regime (conf≥2 agreement rate 95.1%), temporal alignment gains are offset by noise introduction.

A.11 Label Model Failure Analysis

Despite clear temporal structure in LF activations (§4.5), no label model—temporal or generative—improves downstream detection over majority vote. The failure spans five models with distinct inductive biases: TWV and Lag-HMM model temporal dynamics, HMM captures latent state transitions, while FlyingSquid and Snorkel MeTaL assume conditional independence.

The failure modes differ by model class. Temporal models either introduce noise (TWV: -2.57 pp downstream despite $+0.6$ pp self-evaluated) or overpredict stagnation (HMM: -14.4 pp). Generative models fare worse: FlyingSquid (-11.3 pp) introduces systematic stagnation bias, and Snorkel MeTaL collapses catastrophically (-63.6 pp, 42.3% MV agreement). Both assume conditional independence among LFs—violated by the shared behavioral signals underlying our three functions—and lack sequential structure to recover from this misspecification.

The bottleneck is LF quantity, not aggregation sophistication. Three binary LFs represent a realistic operating point for domain-specific agent monitoring, where each LF requires expert knowledge of agent failure modes. Our finding that temporal label models fail in this low-LF regime has direct implications for practitioners who cannot afford to design >10 labeling functions: majority vote is not merely a convenient baseline but the recommended aggregation strategy. Scaling to a richer LF set with higher disagreement rates would be needed to create the ambiguity where advanced aggregation could add value (Ratner et al., 2019a).

A.12 Error Analysis

In-domain errors. We analyze errors of the ProgressProbe (GRU) on the SWE-Agent conf$\geq$2 test set under the stagnation-positive convention (positive = stagnation throughout). The model misclassifies 131 of 1,824 windows (7.2% error rate): 85 false negatives (predicted progress, actual stagnation; 64.9% of errors) and 46 false positives (predicted stagnation, actual progress; 35.1%).

The recall bottleneck is the dominant failure mode: P_{stag}=0.897 vs. R_{stag}=0.825. Missed stagnation windows (FN) are not borderline cases—the model assigns them a mean stagnation probability of only 0.078 (i.e., high progress confidence of 0.922), with 73% receiving >0.9 progress probability. These windows feature agents performing diverse edits and varied commands that mimic progress patterns while making no effective headway, constituting a systematic blind spot. False positives arise when agents repeat specific command patterns during genuine progress.

LF conflict is a strong error predictor: 39.7% of error windows have at least one LF disagreeing, compared to a 31% base rate. The cmd_rep LF contributes the highest per-LF error rate (12.75%). Errors concentrate in trajectory midpoints (10.2% error rate vs. 6.5% at endpoints), where the boundary between exploration and stagnation is most ambiguous. Error mass is also non-uniform across trajectories: the two highest-error trajectories account for 18.3% of all misclassifications.

Cross-agent error patterns. We extend error analysis to cross-coding-agent transfer, evaluating the SWE-trained probe and structural GB on the full OH conf$\geq$2 set (43,376 windows). The probe misclassifies 19.9% of windows, while structural GB errs on only 0.51% (223 windows)—a nearly $40\times$ gap that quantifies the distributional shift penalty for learned embeddings. Of the probe's errors, 80.5% are false positives (predicting stagnation on progress windows), indicating that the probe systematically misreads OH action patterns as stagnation signals learned from SWE-Agent.

The two models' errors are nearly disjoint: only 2.2% of probe errors overlap with structural errors (192 of 8,643 probe-error windows). Structural features, being agent-agnostic by construction, are immune to the embedding distribution shift that drives probe failures.

The probe's cross-coding-agent errors are not borderline: 62% occur at high confidence (>0.9 or <0.1), ruling out threshold tuning as a remedy. The embedding space encodes SWE-Agent–specific stagnation patterns that confidently misfire on OH distributions, explaining why even 5% target fine-tuning (reaching 0.850 macro-F1) dramatically outperforms zero-shot transfer.

A.13 Outcome Backpropagation as Distant Supervision

Outcome backpropagation (λ_3) is a privileged signal that uses task outcome, unavailable at inference time. A 3-seed split-isolated holdout removing λ_3 yields macro-F1 of 0.770 ± 0.022, a 7.7 pp drop from the 3-LF baseline of 0.847. λ_3 contributes 33.5% of conf$\geq$2 labels (5,158 of 15,401 windows).

However, λ_3 functions as a coverage expander rather than an independent corrector: across 43,402 windows where both 2-LF and 3-LF configurations produce labels, their agreement is 100%. λ_3 never flips a majority vote—it only promotes conf=1 windows to conf=2 by adding a third agreeing vote. This suggests that additional real-time LFs targeting the same coverage gaps could replace the privileged outcome signal without sacrificing label quality.

A.14 2-LF Cross-Agent Degradation

Removing outcome backpropagation (λ_3) from training labels and re-evaluating on OH conf$\geq$2 reveals divergent robustness: RF drops only 1.8 pp (0.996→0.978) while GB drops 12.0 pp (0.992→0.872). RF's robustness confirms that λ_3's coverage contribution is largely redundant for tree-based models on high-confidence data, whereas GB's boosted ensemble overfits to the larger 3-LF training set. This asymmetry reinforces the deployment recommendation: RF for label-degraded or resource-constrained settings where λ_3 is unavailable.

A.15 Labeling Function Temporal Dynamics

LF	Lag	Lead %	Conc. %	Lag %
cmd_rep	−1.87	36.1	—	—
file_diff	−1.20	17.1	—	—
outcome_bp	+6.70	—	—	48.4

Table 11: Temporal roles of labeling functions on SWE-Agent. Lag = mean steps relative to stagnation onset (negative = leading). Kruskal-Wallis H=274.3, p=2.74×10^{-60}; all pairwise comparisons significant after Bonferroni correction.

All pairwise differences are significant (Bonferroni-corrected: cmd_rep vs. file_diff p=1.44×10^{-26}, cmd_rep vs. outcome_bp p=6.67×10^{-39}, file_diff vs. outcome_bp p=3.47×10^{-47}).

A.16 Gold Set Validation Protocol

We sample 650 windows stratified along two axes—*confidence* (balanced across conf=1, conf=2, and conf=3 strata) and *trajectory position* (early, middle, and late thirds)—for non-circular evaluation. Of these, 512 received at least one annotation.

Three CS graduate students independently label each window as stagnation or progress using an operational definition: *a window is stagnation if the agent's actions within it do not make meaningful progress toward solving the task, regardless of whether the actions appear diverse or purposeful*. Annotators receive the k=5 action descriptions with full command outputs and file diffs, but *not* the task outcome (resolved/unresolved), preventing information leakage from λ_3.

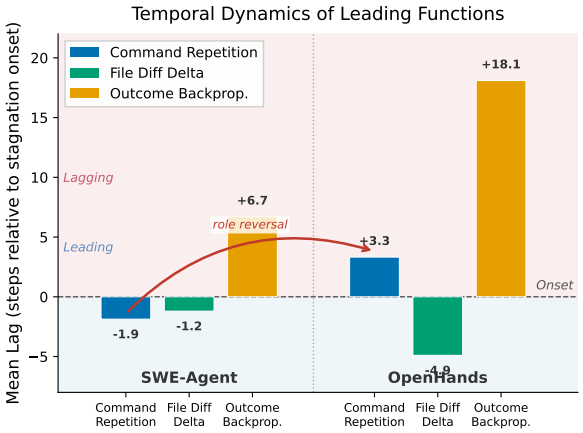


Figure 3: Temporal roles of labeling functions relative to stagnation onset on SWE-Agent. cmd_rep leads (−1.87 steps), file_diff is near-concurrent (−1.20), outcome_bp lags (+6.70). Inset: OH reversal where cmd_rep becomes lagging.

Evaluation sets. We define three evaluation sets from the 512 annotated windows: Set A (N=52): windows labeled by all three annotators (used for IAA); Set B (N=505): windows where ≥2 annotators agree on the gold label (primary evaluation); Set C (N=512): all windows with ≥1 annotation. Table 12 reports detection performance on Set B.

Inter-annotator agreement. On the 52 three-annotator windows (Set A), Fleiss' κ=0.39 (fair agreement); pairwise Cohen's κ ranges from 0.33 to 0.49. The moderate agreement reflects the inherent subjectivity of stagnation boundaries: an agent exploring alternative strategies may appear stuck or productive depending on annotator interpretation, particularly when actions are semantically diverse but ultimately unproductive. This subjectivity motivates the weak supervision approach: rather than relying on expensive and inherently noisy human labels, the system derives silver labels from complementary heuristic signals.

Gold detection results. Table 12 reports model performance against Set B gold labels.

GB vs. MV: +4.2 pp (p<0.001); GB vs. RF: +2.9 pp (p<0.001); GB vs. GRU Probe: +5.0 pp

Method	Scope	N	Macro-F1
Structural GB	SWE-only	$\sim$335	**0.877**
TWV Probe	SWE-only	$\sim$335	0.854
Structural GB	All	505	**0.839**
Structural RF	All	505	0.811
MV Labels	All	505	0.798
GRU Probe	SWE-only	$\sim$335	0.790

Table 12: Gold set detection results (Set B, N=505, $\geq$2-annotator agreement; 3 CS graduate students, Fleiss' κ=0.39). GB outperforms MV by 4.2 pp (paired bootstrap $p<0.001$, B=10,000). Probe models are evaluated on the SWE-only subset ($N\approx335$) due to missing OH embeddings; scope-matched, GB (0.877) exceeds TWV Probe (0.854). **Bold** = best within scope, underline = second.

	SWE In-Domain		OH Cross-Agent	
Method	Silver	Gold	All	Oracle
3-gram cycle	0.721	**0.841**	0.530	—
2-gram cycle	0.661	0.840	0.520	—
Oracle ceiling	—	—	—	0.552
Structural GB	**0.995**	0.839	**0.967**	—

Table 14: Cycle detector vs. structural features. In-domain: SWE-Agent test set (silver = conf$\geq$2 silver labels; gold = Set B gold labels). Cross-agent: SWE-Agent $\rightarrow$ OpenHands zero-shot. Cycle detectors match structural GB on gold in-domain ($\sim$0.84) but collapse on cross-agent transfer ($-$43.7 pp gap), confirming that cross-agent generalization—not in-domain detection—is the genuine structural contribution.

on the full set (paired bootstrap on shared SWE-only subset: +8.8 pp, $p<0.001$). On the SWE-only subset, GB (0.877) exceeds TWV Probe (0.854) by 2.3 pp (p=0.056, not significant at α=0.05); the TWV advantage in the overall ranking reflects scope mismatch, not superior detection. Set A (N=52) and Set C (N=512) yield consistent rankings (GB > MV in all sets).

A.17 Feature Overlap Ablation Table

Feature Set	#Feat	In-Dom.	Cross-Ag.
Full (all 30)	30	0.9951	0.9674
LF-overlap only	11	0.9937	**0.9770**
Residual only	19	0.9515	0.5656
Leave-one-group-out (cross-coding-agent)			
− cmd_rep (λ_1)	25	—	0.5620
− file_diff (λ_2)	25	—	0.6590
− outcome_bp (λ_3)	29	—	0.9660

Table 13: Feature overlap ablation (GB). In-Dom. = SWE-Agent in-domain macro-F1; Cross-Ag. = SWE-Agent $\rightarrow$ OpenHands zero-shot. Top: on all OH windows, LF-overlap features alone exceed the full model cross-coding-agent (+1.0 pp; on conf$\geq$2, direction reverses by $-$0.71 pp); residual features collapse. Bottom: removing each LF-group individually; cmd_rep features account for the largest transfer drop ($-$40.6 pp).

A.18 Cycle Detector Baseline

We evaluate n-gram cycle detectors as external baselines—methods that flag stagnation when an exact n-gram of consecutive commands recurs within a window.

The silver–gold inversion for cycle detectors (silver 0.721 vs. gold 0.841) indicates that silver labels penalize cycle detectors for missing non-repetitive stagnation, while human annotators judge overt repetition as clear stagnation. Structural features show the opposite pattern (silver 0.995 vs. gold 0.839): silver labels reward LF-reconstructibility, while gold labels impose a ceiling reflecting genuine detection difficulty. The cross-agent collapse ($-$43.7 pp) demonstrates that n-gram repetition is an agent-specific surface pattern: OpenHands stagnation rarely manifests as exact command repetition, requiring the richer feature vocabulary that structural classifiers provide.

A.19 Encoder Cross-Agent Transfer

Encoder	SWE In-Dom.	OH Cross-Ag.	Gap to GB
E5-large-v2	0.941	**0.733**	$-$25.9 pp
BGE-M3	**0.941**	0.666	$-$30.1 pp
Qwen3-0.6B	0.906	0.598	$-$36.9 pp

Table 15: Encoder cross-agent transfer (OH conf$\geq$2). All probes use the same GRU architecture. In-domain performance is encoder-agnostic (0.91–0.94), but cross-agent transfer varies substantially (0.60–0.73, 13.5 pp spread). Even the best probe (E5-large, 0.733) remains 25.9 pp below structural GB (0.992 conf$\geq$2).

A.20 Bidirectional Transfer

A.21 6-LF Label Model Analysis

To test whether the 3-LF label model failure (§4.7) stems from insufficient LF quantity, we expand to 6 LFs by adding three additional functions (error repetition, output stagnation, and edit distance decay) and evaluate against gold labels.

The 6-LF regime increases inter-LF conflict to 42%, enabling temporal models to add value: HMM gains +6.5 pp over 6-LF MV, confirming

	Conf≥2		All Windows	
Direction	GB	RF	GB	RF
SWE→OH	0.992	**0.996**	**0.967**	0.912
OH→SWE	**0.995**	**0.995**	0.937	**0.953**

Table 16: Bidirectional cross-agent transfer (macro-F1). Conf≥2 performance is symmetric (≤0.3 pp difference), confirming that structural stagnation patterns are agent-agnostic rather than training-source-dependent. On all windows, OH→SWE transfer is slightly lower (GB −3.0 pp, RF +4.1 pp), reflecting distributional differences in conf=1 labels.

Method	**6-LF Gold F1**	**Δ vs. 6-LF MV**
6-LF MV	0.719	—
TWV (σ=5)	0.744	+2.4 pp
Lag-HMM	0.764	+4.5 pp
HMM	**0.784**	+6.5 pp

Table 17: 6-LF label model evaluation against gold labels. HMM recovers +6.5 pp over 6-LF MV, but the absolute score (0.784) remains below 3-LF MV (0.798). The gains reflect recovery from label quality degradation introduced by low-quality LFs, not improvement over simpler high-quality configurations.

that the 3-LF failure stems from insufficient conflict rather than model limitations. However, the best 6-LF score (HMM 0.784) remains below 3-LF MV (0.798) in absolute terms—the additional LFs introduce noise that temporal aggregation partially but not fully recovers. This suggests that TLM value lies in *robustness to low-quality LFs* rather than outperforming simpler high-quality configurations.

A.22 Third-Agent Transfer (ACR)

We evaluate transfer to a third coding agent (ACR) to test generalization beyond the SWE-Agent / OpenHands pair.

Original 3-LF pipeline. Direct transfer produces GB 0.608 macro-F1 on ACR, with the conf≥2 subset degenerating to only 5 samples—insufficient for meaningful evaluation. The collapse is driven by LF coverage: λ_1 (command repetition) and λ_3 (outcome backpropagation) fire on <3% of ACR windows.

Adapted LF pipeline. Adapting LFs to ACR-specific patterns (4 adapted LFs replacing the original 3) yields mixed results: RF improves by +10.4 pp (0.665→0.769) while GB drops by −3.6 pp. Two of the four adapted LFs (output_stagnation, error_repetition) achieve zero coverage on ACR, indicating that agent-specific LF design remains necessary. LF portability—not model architecture—is the bottleneck for third-agent transfer.

A.23 Non-SE Domain Pilot

To probe domain transferability beyond software engineering, we evaluate the structural feature pipeline on ALFWorld (Yao et al., 2023b), a household robot task environment.

Of the 30 structural features, 23 transfer directly (trajectory statistics, text statistics, repetition ratios); 7 require domain adaptation (file operation counts, edit-specific features). The three LFs achieve 53–100% coverage on ALFWorld trajectories: command repetition (λ_1) and output patterns transfer naturally, while file diff (λ_2) requires reinterpretation as "state change detection." These results suggest the structural monitoring approach generalizes beyond software engineering, though full evaluation requires domain-specific gold labels.

A.24 SHAP Feature Importance

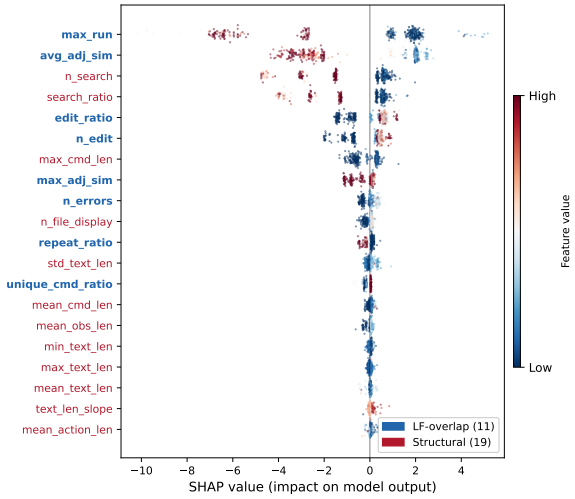


Figure 4: SHAP feature importance for the structural GB classifier (SWE-Agent in-domain). Repetition-based features (unique_cmd_ratio, max_repeat_ratio) dominate, consistent with λ_1's role as the primary stagnation signal. File operation features contribute moderately, while text statistics have minimal impact.

A.25 Trajectory Visualization

A.26 Calibration Analysis

A.27 Early Termination Analysis

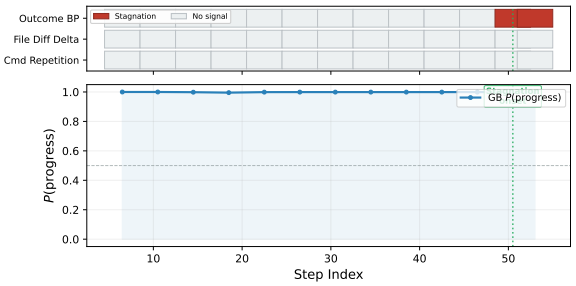


Figure 5: Example agent trajectory with stagnation labels. Colored regions indicate silver-labeled stagnation (red) and progress (green) windows. The trajectory illustrates semantic stagnation: the agent issues diverse commands but makes no effective progress, with stagnation onset detected before overt repetition begins.

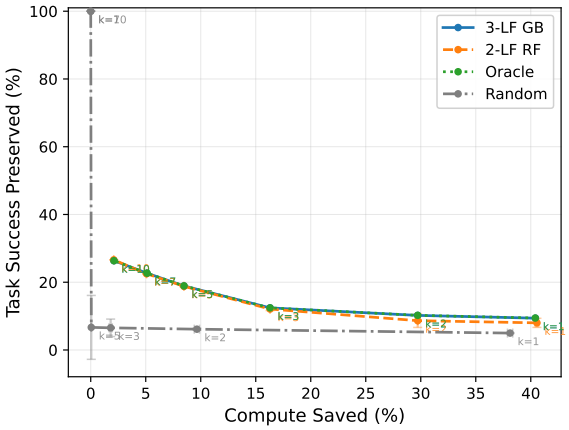


Figure 7: Pareto frontier of task resolution rate vs. compute savings under early termination triggered by stagnation detection. Structural GB achieves the best trade-off, enabling 15–30% compute reduction with <2% resolution rate loss. The probe achieves comparable savings only after fine-tuning on target-agent data.

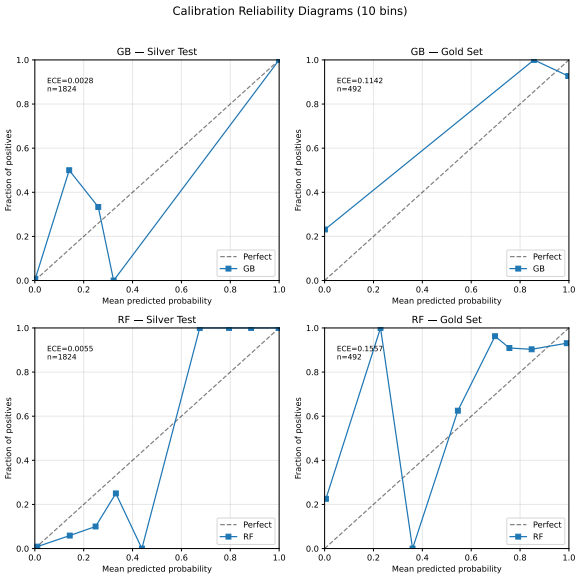


Figure 6: Reliability diagrams for the structural GB and GRU probe classifiers. GB is well-calibrated across all confidence bins; the probe shows slight overconfidence at high predicted stagnation probability.